

A Appendix

A.1 Translating Pseudocode to Each Language

The translation from pseudocode to each language is straightforward:

- (1) Function calls translate into function calls
- (2) **spawn** is translated into a **spawn** for runtimes that have it (Asyncio, Trio, Tokio, Smol) and a *noop* for languages that don't (C#, JavaScript, Swift)
- (3) All throwing function are assumed to have a **try** in front of them following Swift semantics

A.2 Design Documents Describing Straight-Line Asynchrony

(JavaScript) With async functions, all the remaining boilerplate is removed, leaving only the semantically meaningful code in the program text. [1]

(Swift) Modern Swift development involves a lot of asynchronous (or “async”) programming using closures and completion handlers, but these APIs are hard to use. This gets particularly problematic when many asynchronous operations are used, error handling is required, or control flow between asynchronous calls gets complicated. This proposal describes a language extension to make this a lot more natural and less error prone. [25]

A.3 Additional Implementations of timeout

JavaScript

```

1  function sleepCancellable(duration: number, opts?: { signal?: AbortSignal }): Promise<null> {
2    let timerId: number;
3    let signalHandler: () => void;
4    const timer = new Promise<null>((accept, reject) => {
5      signalHandler = () => reject(opts?.signal?.reason);
6      opts?.signal?.addEventListener("abort", signalHandler, { once: true });
7      timerId = setTimeout(() => accept(null), duration);
8    })
9
10   return timer.finally(() => {
11     clearTimeout(timerId);
12     opts?.signal?.removeEventListener("abort", signalHandler);
13   });
14 }
15
16 async function timeoutCancellable<T>(
17   duration: number,
18   controller: AbortController,
19   f: (signal: AbortSignal) => Promise<T>,
20 ): Promise<T | null> {
21   const result = await Promise.race([
22     sleepCancellable(duration, { signal: controller.signal }),
23     f(controller.signal)
24   ]);
25   if (result === null) controller.abort();
26   return result;
27 }

```

Rust+Smol

```

1324 1  async fn timeout_task<T, Fut>(duration: Duration, task: Fut) -> Option<T>
1325 2  where
1326 3      T: Send + 'static,
1327 4      Fut: Future<Output = T> + Send + 'static,
1328 5  {
1329 6      let mut handle = smol::spawn(task);
1330 7      let result = (&mut handle)
1331 8          .map(Some)
1332 9          .or(sleep(duration).map(|_| None))
1333 10         .await;
1334 11      if result.is_none() {
1335 12          handle.cancel().await;
1336 13      }
1337 14      result
1338 15  }

```

A.4 Eagerness Programs

<pre> 1345 1 async fn work(msg: &str) { 1346 2 print!("{msg}"); 1347 3 sleep(one_ms).await 1348 4 } 1349 5 1350 6 #[tokio::main] 1351 7 async fn main() { 1352 8 let a = work("A"); 1353 9 let b = work("B"); 1354 10 print!("C"); 1355 11 a.await; b.await; 1356 12 } // => CAB </pre>	<pre> 1 func work(msg: String 2) async throws { 3 print(msg) 4 try await Task.sleep(5 for: .milliseconds(1)) 6 } 7 8 async let a = work(msg: "A") 9 async let b = work(msg: "B") 10 print("C") 11 try await a; try await b 12 // => CAB, CBA, BCA, ACB, ... </pre>	<pre> 1 async Task work(string msg) { 2 Console.Write(msg); 3 await Task.Delay(1); 4 } 5 6 async Task Main() { 7 var a = work("A"); 8 var b = work("B"); 9 Console.Write("C"); 10 await a; await b; 11 } // => ABC </pre>
--	---	--

(a) Rust function applications are lazy. The function work does not execute until line eleven. The print order of the program is always “CAB” because a, and b are not running concurrently.

(b) Swift function applications are semi-eager. The right-hand-side of an async let binding is evaluated in a concurrently executing task, execution of the current thread does not transfer to the body of the applied function. The print order of the program is any permutation of the string “ABC,” though typically it will begin with ‘C.’

(c) C# function applications are eager. When work is called the executing thread transfers control to the body of the async function until it is suspended. The print order of this program is always “ABC”.

Fig. 18. Examples of lazy, semi-eager, and eager async function application.

A.5 Suspension Programs

```

1  async function print(msg) {
2    console.log(msg);
3  }
4
5  async function work(name) {
6    await print(name);
7    await print(name);
8    await print(name);
9  }
10
11 async function main() {
12   let a = work("A");
13   let b = work("B");
14   await print("C")
15   await a; await b
16 }
17
18 await main()
19 // => ABCABAB

```

(a)

```

1  async fn print(msg: &str) {
2    print!("{msg}");
3  }
4
5  async fn work(name: &str) {
6    print(name).await;
7    print(name).await;
8    print(name).await;
9  }
10
11 fn main() {
12   smol::block_on(async {
13     let a = smol::spawn(work("A"));
14     let b = smol::spawn(work("B"));
15     print("C").await;
16     a.await; b.await;
17     drop(signal);
18   });
19 } // => CAAABBB

```

(b)

Fig. 19. Examples of static and dynamic suspension.

A.6 Extent Programs

```

1400 func innerWork() async {
1401   try? await Task.sleep(
1402     for: .milliseconds(100))
1403   print("done")
1404 }
1405
1406 func work() async throws {
1407   async let a /* T1 */ = innerWork()
1408   try await Task.sleep(
1409     for: .milliseconds(1))
1410   print("exiting")
1411 }
1412
1413 try await work()

```

(a) Tasks have dynamic extent in Swift. References to the task T_1 cannot be made after its calling construct, the work function outlined in gray, terminates — including references made by the runtime system. The program prints “exiting” and then “done.”

```

public static async Task InnerWork() {
    await Task.Delay(100);
    Console.Write("done");
}

async Task Work() {
    var a /* T1 */ = InnerWork();
    await Task.Delay(1);
    Console.Write("exiting");
}

async Task Main() {
    await Work();
}

```

(b) Tasks have indefinite extent in in C#. Task T_1 may continue to be referenced after the Work function has terminated. References made by the runtime are strong references. This program prints “exiting,” and the print in InnerWork is never executed.

Fig. 20. Examples of dynamic and indefinite extent.

A.7 Destruction Programs

```

1  async function inner() {
2    await new Promise(r =>
3      setTimeout(r, 10_000))
4    console.log("done")
5  }
6
7  async function work() {
8    let a = inner()
9    await new Promise(r =>
10      setTimeout(r, 0))
11    console.log("exiting")
12  }
13
14  await work() // ~10s

```

(a) JavaScript awaits all running tasks at the end of their scope (program scope). The execution of the above program takes about 10 seconds and prints “exiting,” and “done,” because the leaked task is fully awaited.

```

1  func inner() async {
2    try? await Task.sleep(
3      for: .seconds(10))
4    if Task.isCancelled {
5      print("cancelled")
6    }
7    print("done")
8  }
9
10 func work() async throws {
11   async let _ = inner()
12   try await Task.sleep(
13     for: .seconds(0))
14   print("exiting")
15 }
16
17 try await work() // ~0.5s

```

(b) Swift cancels all running tasks on destruction. The execution of the above program takes about 0.5 seconds and prints “exiting,” “cancelled,” and “done.”

```

1  async Task InnerWork() {
2    await Task.Delay(10_000);
3    Console.Write("done");
4  }
5
6  async Task Work() {
7    var _ = InnerWork();
8    await Task.Delay(0);
9    Console.Write("exiting");
10 }
11
12 async Task Main() {
13   await Work();
14 } // ~0.5s

```

(c) C# terminates all running tasks at the end of their scope (program scope). The execution of the above program takes about 0.6 seconds. The inner Task is terminated and the sleep terminates early with no warning.

Fig. 21. Examples of await, cancel, and termination destruction semantics.

A.8 Semantics

See the provided supplemental for the Redex model. A fully typeset version of the rules can be provided at a later time.

A.9 Zig async/await

```

1  const std = @import("std");
2  const Io = std.Io;
3  const Allocator = std.mem.Allocator;
4  const assert = std.debug.assert;
5
6  fn producer(
7      io: Io,
8      queue: *Io.Queue([]const u8),
9      flavor_text: []const u8,
10 ) !void {
11     try queue.putOne(io, flavor_text);
12 }
13
14 fn consumer(
15     io: Io,
16     queue: *Io.Queue([]const u8),
17 ) ![]const u8 {
18     return queue.getOne(io);
19 }
20
21 fn run_client_server(io: Io) !void {
22     var queue: Io.Queue([]const u8) = .init(&.{});
23
24     var producer_task = try io.concurrent(producer, .{
25         io, &queue, "hello",
26     });
27     defer producer_task.cancel(io) catch {};
28
29     var consumer_task = try io.concurrent(consumer, .{ io, &queue });
30     defer _ = consumer_task.cancel(io) catch {};
31
32     const result = try consumer_task.await(io);
33     std.debug.print("msg received {s}\n", .{result});
34 }

```

Fig. 22. An example of Zig's unstabalized async/await features. Zig code is now parameterized over an implementation of the Io interface. Clients can instantiate the interface with a threaded, asynchronous, concurrent, or synchronous implementation.